

CE-Replay: Executable Cardinality-Estimation Semantics for Statistics Physical Design

AUTHOR METADATA PENDING

AFFILIATION METADATA PENDING

CITY METADATA PENDING, COUNTRY METADATA PENDING

EMAIL-METADATA-PENDING@example.invalid

ABSTRACT

Selecting extended statistics for a supplied target workload is a physical-design problem with two distinct constraints: statistics can interact non-monotonically through the estimator, and deployed objects consume recurring maintenance capacity. Independent candidate scores are therefore insufficient because native applicability, winner selection, clause consumption, and mechanism composition make each candidate’s effect contextual. We introduce CE-Replay, a workload-specialized, design-parametric representation that keeps the supported statistics-sensitive estimator semantics executable and exposes both an objective oracle and a semantic dependency oracle. We instantiate CE-Replay for PostgreSQL 16.14 conjunctive base-relation restrictions with MCV, functional-dependency, and constant IN/= ANY MCV semantics, and use it with deterministic maintenance-constrained local search. Across sparse Census and dense IN-heavy DMV, CE-Replay matches fresh native PostgreSQL estimates on 468/468 and 1,965/1,965 queries, supports mixed-mechanism designs, and preserves audited incremental move decisions while reducing semantic replay work; the mixed evaluator also shows that less semantic work need not yield lower wall-clock time. These results establish that executable native semantics can support statistics physical design within a validated fragment, while broader CE coverage, hypothetical-payload acquisition, realization-robust selection, and runtime objectives remain open.

PVLDB Reference Format:

AUTHOR METADATA PENDING. CE-Replay: Executable Cardinality-Estimation Semantics for Statistics Physical Design. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/1951123/ce-replay>.

1 INTRODUCTION

Cost-based query optimization depends on estimates of how many tuples flow through candidate plans. Ordinary per-column statistics can be inadequate when predicates are correlated, so DBMSs provide extended statistics that expose multivariate information to the

estimator. Deciding which extended statistics to maintain changes the information available during optimization and is therefore a physical-design decision, not merely a statistics-collection setting. This paper studies that decision for a supplied target workload.

Selection is semantically necessary because a DBMS may choose among overlapping statistics, consume clauses after using one statistic, compose statistics over disjoint dimensions, and suppress a downstream mechanism after an upstream mechanism changes the residual state. A candidate’s effect therefore depends on the other selected objects and their physical realization. In both Census and DMV, some additions worsen the target-workload cardinality-estimation objective; even with loose capacity, selecting every statistic is not a valid default in these settings.

Selection also has an independent resource justification. PostgreSQL must collect and refresh deployed statistics, and this recurring ANALYZE work depends on the mechanism and environment. A practical design must balance estimator utility against maintenance capacity: a statistic may be affordable yet harmful, or beneficial yet infeasible.

Automatic statistics management and workload-driven selection have a substantial history, as do what-if physical-design interfaces and reusable representations of optimizer response [2–4, 10]. Combinatorial statistics design nevertheless requires evaluating many hypothetical states whose effects are generated inside the estimator. Efficient move evaluation in this setting benefits from tracking how a statistics-design change can alter estimator applicability, clause consumption, numerical contribution, and downstream mechanism state. The challenge addressed here is therefore not hypothetical evaluation in general, but retaining the relevant native estimator computation and its design-relevant dependencies across hypothetical statistics states.

Our key idea is to specialize the DBMS’s supported statistics-sensitive estimator semantics to the supplied workload while leaving statistics-design-dependent behavior executable. Workload-fixed clauses, relation context, and ordinary selectivities become fixed context; candidate payloads remain explicit inputs; and applicability, winner selection, clause consumption, mechanism composition, and numerical updates execute for each hypothetical design. The result, CE-Replay, produces the DBMS-style cardinality estimate. Ground truth enters only at the design layer, where the estimate is converted into q-error and a workload objective.

Executable semantics provide two interfaces. The **objective oracle** evaluates the estimate and loss induced by a hypothetical physical statistics state without learning a separate subset-to-q-error response model inside the supported fragment. The **dependency oracle** identifies the queries, semantic dimensions, and downstream mechanisms that a design move can affect. Because both interfaces

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

arise from the same executable state transitions, the optimizer can avoid semantically irrelevant recomputation while preserving audited move values.

We instantiate CE-Replay for the supported PostgreSQL 16.14 statistics-sensitive fragment of conjunctive base-relation restrictions. The implementation covers validated scalar predicates, constant IN and = ANY MCV predicates, multicolumn most-common-values statistics, functional dependencies, PostgreSQL precedence relevant to MCV ties, and directed MCV-to-FD composition. It is source-informed and validated against native instrumentation; it is not a complete PostgreSQL CE emulator or an automatic source compiler.

We evaluate two structurally contrasting workloads: Census has a large candidate universe with sparse candidate-query incidence, while DMV has a small, dense, high-reuse universe dominated by constant IN predicates. Within the supported realization boundary, replay matches native raw estimates to floating-point tolerance, including all 468 fresh Census queries and all 1,965 fresh DMV queries after deployment. The experiments establish non-monotone utility, mechanism-dependent maintenance cost, mixed MCV+FD selection, and audited incremental equivalence. The evidence does not make the workloads representative, the workload-scale search globally optimal, or reduced semantic work a guarantee of lower runtime.

This paper makes four contributions:

- (1) **Problem and representation.** We connect maintenance-constrained statistics design with workload-specialized, design-parametric execution of the supported native CE semantics. The formulation separates selected definitions, DBMS-specific physical realization, frozen payloads, and fresh realization.
- (2) **PostgreSQL CE-Replay.** We implement and validate CE-Replay for the supported PostgreSQL 16.14 base-restriction MCV+FD fragment, including bounded constant ScalarArray semantics and directed mechanism composition. The scope is explicit and excludes arbitrary PostgreSQL CE.
- (3) **Semantics-guided optimization.** We use CE-Replay as a validated objective oracle and a statistics-semantic dependency oracle for deterministic maintenance-constrained ADD/DROP/SWAP search, including incremental move evaluation that reproduces audited full-replay move values within the supported setting. Workload-scale guarantees are neighborhood-local rather than global.
- (4) **Cross-workload physical validation.** We demonstrate contextual non-monotonicity, maintenance-aware mixed designs, dependency-aware evaluation, and fresh physical deployment on sparse Census and dense IN-heavy DMV. The evidence concerns CE loss and semantic fidelity, not query-runtime improvement.

2 BACKGROUND AND MOTIVATION

2.1 Extended statistics and cardinality estimation

A query optimizer estimates predicate selectivity and intermediate cardinality to compare candidate plans. Per-column statistics commonly induce an independence approximation for multiple

predicates; correlations can make that approximation inaccurate. Work on correlation discovery and multivariate summaries addresses this gap, and PostgreSQL exposes functional-dependency and MCV extended statistics for selected attribute groups [7, 12].

PostgreSQL supports several extended-statistics mechanisms. The two used here have different semantics. A multicolumn MCV object stores frequent value combinations and frequencies, allowing compatible predicates to be tested directly against a joint distribution. A functional-dependency object records approximate dependency strengths and adjusts the treatment of equality-eligible predicates. Section 4 describes only the control and numerical behavior required by CE-Replay.

2.2 Statistics as physical design

A candidate statistic specifies a persistent definition that may be deployed and refreshed. Unlike an index, it does not primarily add an access operator; it changes the estimator’s information state and hence the estimates used during plan search. Workload-aware statistics management, selective statistics collection, and resource-constrained physical design are established research areas [1, 4, 5]. The selected design studied here additionally depends on native statistics-consumption semantics and recurring maintenance work.

Candidate definition, payload, and realization are distinct. A definition identifies what PostgreSQL should collect; a payload contains sampled values, frequencies, or dependency degrees; and a physical realization includes implementation state such as relevant catalog precedence. Re-running ANALYZE can change payload values or availability without changing the definition. Just-in-time, piggyback, and incremental collection address the complementary question of when and how to acquire or refresh statistics [5, 11, 15].

2.3 Why independent candidate scoring fails

Consider two overlapping MCV objects and one FD object. Adding the larger MCV object may cause it to win native selection, consume clauses that previously supported the smaller object, and suppress the FD stage. The same addition can help one query and hurt another, and its effect can change after another candidate is selected. A fixed singleton score cannot encode that state transition. The non-monotonicity measured in Section 8 makes this semantic concern operational rather than hypothetical.

2.4 Design requirements

The problem calls for six properties. Hypothetical evaluation must match native behavior within a stated semantic boundary; design-dependent control must remain executable; mechanisms must compose through explicit state; repeated evaluation should exploit safe dependencies; maintenance must be an explicit resource; and a selected state must be deployable and validated against fresh native behavior. CE-Replay addresses the representation and evaluation requirements. The deterministic local search in Section 6 is one replaceable consumer of its interfaces.

3 PROBLEM FORMULATION

We consider offline physical design for a supplied target workload. The inputs are a database instance, queries with ground-truth cardinalities, candidate-statistic definitions with a frozen payload

repository, and a recurring maintenance budget. The output is a realizable physical statistical state minimizing cardinality-estimation loss on that workload. The workload is a direct input to physical design rather than data for fitting a learned estimator.

Let the target workload, one query, and its true cardinality be denoted by

$$Q, \quad q \in Q, \quad N_q.$$

For each query, workload-fixed context is denoted by

$$W_q.$$

It contains clauses, relation context, ordinary selectivities, and other design-independent inputs required by the supported estimator fragment. Ground truth scores estimates but is not consumed by CE-Replay.

Let the candidate universe and selected subset be

$$S, \quad Y \subseteq S.$$

Selection need not fully determine the DBMS state. A DBMS-specific physical realization is denoted by

$$\rho.$$

For PostgreSQL, relevant creation/catalog precedence can break tied MCV choices. This does not imply that every DBMS design universally includes a permutation.

Candidate definitions do not uniquely determine payloads. The frozen candidate-payload repository used for hypothetical evaluation is

$$P.$$

It includes mechanism values and the schema needed to interpret them, including semantic key order and availability. The current system assumes that an offline acquisition process populated this repository. A fresh deployment can generate a different realization.

CE-Replay's executable semantics are denoted by

$$F.$$

For one query, the replayed estimate is

$$\widehat{N}_q(Y, \rho; P) = F(W_q, Y, \rho, P).$$

CE-Replay is a workload-specialized, design-parametric executable representation of a supported statistics-sensitive native CE fragment. It is neither a learned predictor nor a table of complete-design responses: eligibility, selection, consumption, composition, and numerical updates remain executable.

The per-query loss is denoted by

$$\ell.$$

The evaluated q-error is

$$\ell(\widehat{N}, N) = \max\left(\frac{\widehat{N}}{N}, \frac{N}{\widehat{N}}\right),$$

with the implementation's documented positive floor for zero values. The unweighted target-workload objective is

$$L(Y, \rho; P) = \sum_{q \in Q} \ell(\widehat{N}_q(Y, \rho; P), N_q).$$

Recurring maintenance cost and its budget are

$$C(Y, \rho), \quad B.$$

The constrained physical-design problem is

$$\min_{Y, \rho} L(Y, \rho; P) \quad \text{subject to} \quad C(Y, \rho) \leq B.$$

This notation defines a mathematical optimum, not a guarantee of the workload-scale algorithm. The evaluated solver fixes recorded precedence and terminates at a local optimum of an audited ADD/DROP/SWAP neighborhood. The objective is CE loss, not query latency, plan quality, or throughput.

4 STATISTICS-SENSITIVE CE SEMANTICS

4.1 Running example

Consider a conjunctive query with predicates over four attributes. A design contains an MCV object over three attributes, an overlapping MCV object over two of them, a disjoint MCV object, and an FD whose applicability depends on residual predicates. At the start, both overlapping MCV objects may be applicable. GreedyCover selects the object covering more currently unestimated semantic dimensions, and the selected object consumes its covered clauses. The smaller overlap may then cease to apply, while the disjoint object remains available in a later round. The FD stage receives this residual state and may be suppressed if its required predicate was consumed.

The example shows why candidate utility cannot be assigned independently. The selected subset determines a sequence of native state transitions rather than a sum of fixed corrections.

4.2 MCV applicability, selection, and consumption

Within the supported fragment, MCV applicability depends on compatible clauses for sufficient semantic dimensions. Compatibility includes predicate/operator form and payload schema, not merely textual column overlap. Supported predicates include the validated scalar forms and constant IN/= ANY forms described below.

PostgreSQL GreedyCover proceeds in validated rounds [13]. It first maximizes coverage of currently unestimated dimensions, then prefers fewer statistic keys, and finally retains relevant catalog/list order for a remaining tie. The winner evaluates its compatible clauses, consumes them, and leaves a residual state for later rounds. Disjoint MCV objects can contribute later; overlapping objects may become inapplicable. A faithful evaluator must therefore execute repeated selection rather than choose one global statistic.

For a selected MCV object, PostgreSQL tests clauses against MCV entries [12, 13]. Matching entries contribute stored and base frequencies, while residual simple selectivity accounts for distribution mass outside the list. CE-Replay executes this payload-dependent rule from values, frequencies, base frequencies, and schema meta-data. It does not replace it with a learned factor.

4.3 Constant ScalarArray predicates

For supported constant IN and = ANY predicates, each relevant MCV item is compared with array elements using OR semantics. Duplicate values do not multiply the frequency of one matched item, and null elements do not create a match. Direct item evaluation preserves native bitmap and frequency behavior; expanding the array into independently estimated scalar equalities would not.

This bounded extension covers the canonical DMV predicate boundary. It does not support arbitrary ScalarArray operators, non-constant arrays, general OR/NOT trees, arbitrary expressions, joins, or parameterized clauses.

4.4 Functional dependencies and directed composition

FD processing is not another GreedyCover instance. PostgreSQL identifies dependency entries compatible with residual equality-eligible predicates, aggregates their degrees, and adjusts selectivity while accounting for implied attributes [13]. Applicability depends on predicate form, semantic dimensions, payload availability, and the state entering the FD stage.

The supported mechanisms compose in a directed order:

```
MCV applicability and GreedyCover
    v residual estimated-clause state
FD applicability and dependency application
```

An MCV winner can therefore suppress an FD that would otherwise apply. Conversely, removing or replacing an MCV can restore FD reachability. This directed state boundary is both an optimization interaction and a dependency boundary for incremental evaluation.

4.5 Consequence for physical design

Changing one candidate can alter applicability, MCV winners, consumed clauses, later MCV rounds, FD reachability, and payload-derived numerical contributions. Candidate response need not be additive and can be non-monotone. The physical-design evaluator must execute these design-parametric semantics; Section 5 describes their workload-specialized representation.

5 CE-REPLAY

CE-Replay is the conceptual center of the method. It turns the supported statistics-sensitive estimator fragment into an executable representation evaluated repeatedly under hypothetical physical states. It is not the search algorithm, the complete DBMS, or the native estimator itself.

The representation is not merely a second implementation of one fixed cardinality calculation. PostgreSQL normally resolves the estimator under one concrete catalog and statistics state; CE-Replay preserves selected parts of that native computation as explicit functions of hypothetical physical statistics states. Design-dependent applicability, selection, consumption, and composition remain executable rather than being frozen into one observed estimate or planner execution. This manually source-informed, bounded, design-parametric form supplies both the objective interface and the statistics-semantic dependency interface used by physical-design search.

Figure 1 separates the two loops needed to interpret the method correctly. The upper loop combines frozen workload context and an offline frozen payload repository to evaluate hypothetical designs and return objective values and semantic dependencies to search. Only the selected state enters the lower loop, where deployment creates definitions in recorded order, fresh ANALYZE produces a new payload realization, and CE-Replay is compared with native PostgreSQL on that same realization.

5.1 What is frozen and what remains executable

The representation has four blocks:

- (1) **Workload-fixed context:** clauses, relation cardinality, ordinary selectivities, predicate semantics, and truth retained only for later scoring.
- (2) **Payload and schema context:** MCV values and frequencies, base frequencies, FD degrees, semantic dimensions, key order, and payload availability.
- (3) **Design-dependent state:** the selected definitions and relevant physical realization, including recorded PostgreSQL precedence.
- (4) **Executable semantics:** applicability, GreedyCover, consumption, MCV numerical combination, FD application, and final row computation.

Specialization fixes the first block and loads the second, while the third remains an input and the fourth executes for every hypothetical state. Freezing a winner observed under one design would violate design-parametric replay. A response table over complete designs would also discard the state transitions needed for counterfactual dependency analysis.

5.2 Supported boundary and validation model

The validated boundary is PostgreSQL 16.14 conjunctive base-relation restrictions with supported scalar predicates, constant IN/= ANY MCV predicates, MCV, equality-eligible FD, MCV-first composition, and relevant fixed precedence. CE-Replay does not cover joins, complete planner/path search, arbitrary expressions or operators, general Boolean trees, all statistics mechanisms, or other PostgreSQL versions.

Native instrumentation exposes the supported estimator at a raw/pre-clamp boundary. Comparisons match query, design, precedence, and payload realization, avoiding integer plan-row rounding as apparent semantic error. The native system is a semantic validation oracle; it is not invoked for every hypothetical state in the final search loop.

5.3 Objective and dependency interfaces

The objective oracle returns the replayed estimate and, after ground-truth scoring, the target-workload loss. It is exact only relative to the validated semantics and supplied realization. It does not predict truth.

The dependency oracle reports which queries, semantic dimensions, and downstream mechanisms a design move may affect. It distinguishes **realized dependencies**, exercised in the current trace, from **structural/counterfactual dependencies**, which can change a future trace even when current outputs coincide. State

Two-loop CE-Replay architecture placeholder
Frozen hypothetical design above; fresh deployment and native validation below

Figure 1: CE-Replay physical-design pipeline. For a supplied target workload, workload-fixed estimator context and an offline frozen candidate-payload repository specialize the supported PostgreSQL 16.14 statistics-sensitive base-restriction fragment while leaving design-dependent MCV selection, clause consumption, FD applicability, and numerical updates executable. CE-Replay exposes an objective oracle and a semantic dependency oracle to maintenance-feasible ADD/DROP/SWAP search. The selected physical state is then created with its recorded PostgreSQL realization and analyzed afresh. Fresh payloads feed both CE-Replay and native PostgreSQL for same-realization semantic validation; equality between frozen and fresh payloads is not assumed.

sufficient to continue one execution is not necessarily safe for arbitrary extension; current-output equivalence is not counterfactual equivalence.

5.4 Frozen and fresh realizations

Optimization is conditional on the frozen payload repository. Deployment invokes ANALYZE and creates a fresh realization whose values or availability may differ. **Semantic replay error** is the replay/native discrepancy for one matched realization. **Payload realization drift** is the difference between frozen and fresh outcomes. Figure 1 separates these quantities, and Section 8 reports them separately.

6 SEMANTICS-GUIDED PHYSICAL DESIGN

The optimizer is one replaceable consumer of CE-Replay’s two interfaces, not the primary contribution. The implementation below tests whether those interfaces suffice for a complete maintenance-constrained search loop.

6.1 Search policy

The solver first builds a maintenance-feasible design by repeatedly recomputing contextual loss reduction per maintenance unit. It then enumerates all feasible ADD, DROP, and SWAP moves under fixed recorded precedence. Each round accepts the deterministic best strict improvement and terminates when none exists.

The result is locally optimal only under this audited neighborhood, payload repository, budget, and fixed precedence. Exhaustive global agreement is established only on a five-candidate Census instance and four restricted DMV instances; no approximation guarantee is claimed.

6.2 Safe incremental evaluation

Candidate-query incidence supplies an outer affected set. Within an affected query, the evaluator chooses the narrowest safe action: no work, a cached numerical update, local MCV control replay, FD replay after direct or upstream invalidation, or broader replay fallback. A computation is skipped only when structural/counterfactual dependencies show that the move cannot affect it.

Global graph connectivity does not invalidate this strategy. A candidate can affect few queries even when chains of shared candidates place nearly the whole workload in one connected component. Conversely, a shared maintenance budget prevents disjoint query neighborhoods from becoming completely independent optimization problems because candidates can exchange budget through SWAP moves.

Reduced semantic work is not synonymous with runtime improvement. Once control replay is avoided, numerical aggregation and move-loop overhead may dominate. Section 8 therefore reports exact trajectory preservation, replay-operation reduction, evaluator latency, and optimizer wall-clock separately.

6.3 Deployment boundary

After search, selected definitions are created in recorded order and analyzed afresh. Fresh payloads are passed both to CE-Replay and native PostgreSQL for same-realization validation. Candidate-payload acquisition remains an offline input boundary and is not counted as recurring deployed maintenance.

7 EXPERIMENTAL METHODOLOGY

The evaluation asks four questions: whether replay matches native semantics, supports constrained design, preserves move values

```

design <- empty
while a feasible candidate has positive contextual benefit per maintenance unit:
  add the deterministic best candidate and update replay state

repeat:
  best <- none
  for each feasible ADD, DROP, and SWAP:
    affected <- dependency_oracle(design, move)
    value <- objective_oracle.incremental(design, move, affected)
    best <- deterministic_best_improvement(best, value)
  if best is none or not a strict improvement:
    return design
  commit(best)

```

while reducing work, and keeps interacting mechanisms faithful after deployment. All semantic claims target PostgreSQL 16.14.

Table 1 establishes the complementary workload regimes used throughout the evaluation.

Census stresses a large pair universe, mixed-mechanism selection, sparse invalidation, and realization analysis. DMV contains 1,913 IN-bearing queries and stresses bounded ScalarArray semantics, dense candidate reuse, independent maintenance calibration, and second-workload deployment. Its small candidate universe does not establish Census-scale search scalability.

Structural pairs induce mechanism-specific MCV and FD definitions. Census has 2,253 pair MCV definitions and 758 usable query-applicable FD payloads in the mixed realization. DMV has 36 pairs; its optimization realization has 36 usable MCV and 34 usable FD payloads. Baseline, optimization, and fresh DMV realizations are kept separate; absolute losses are compared only within their originating realization.

Ground-truth rows score replayed estimates using the q-error in Section 3. Two DMV queries have zero true cardinality, and the frozen positive floor makes them dominate raw aggregate DMV loss. We preserve the completed objective, use nonzero-truth loss only diagnostically, and never compare raw Census and DMV loss magnitudes.

Recurring maintenance is instantiated with mechanism-weighted object counts fitted independently to aggregate ANALYZE latency. **Figure 2** uses separate workload panels because absolute costs and fitted coefficients are environment-specific.

The controls align with those questions. Raw native comparisons test semantic fidelity; exhaustive five-candidate Census and four restricted DMV searches test small-instance optimization; complete terminal ADD/DROP/SWAP audits establish neighborhood-local termination; and incremental-versus-control trajectories test move equivalence. Deployment then evaluates fresh same-realization fidelity separately from frozen-to-fresh drift.

8 EVALUATION

8.1 RQ1 — Does replay match native PostgreSQL?

RQ1 asks whether CE-Replay reproduces native PostgreSQL estimates inside the supported fragment. **Table 2** combines controlled semantic cases, real workloads, and fresh deployments.

Raw instrumentation removes rounded plan rows as an observation artifact. ScalarArray support retains all 128 scalar regression cases while covering the canonical DMV workload. Fresh deployment is the strongest closure because new payloads are generated: CE-Replay matches native estimates for every fresh query in both workloads.

Answer to RQ1. Within the explicitly supported PostgreSQL 16.14 statistics-sensitive base-restriction fragment and matched payload realization, CE-Replay matches native estimates to the reported floating-point tolerances. This result does not cover joins, full planner search, arbitrary predicates, other mechanisms, or other PostgreSQL versions.

8.2 RQ2 — Does replay drive useful resource-constrained design?

RQ2 first asks whether selection remains necessary before capacity becomes binding. **Figure 3** answers that question by separating semantic harm from maintenance scarcity.

In Census, empty, all-statistics, and optimized-subset losses are 11,808.960379, 10,932.295550, and 805.316472 within the shared frozen realization. In the DMV baseline realization, all-mixed equals all-MCV because MCV consumption suppresses every usable FD. These results establish semantic necessity; **Figure 2** independently establishes recurring resource demand.

Having established semantic need, **Table 3** asks what the optimizer selects under the maintenance constraint. DMV raw loss is retained for consistency but is dominated by the two zero-truth queries.

For Census, changing only the resource semantics from the earlier byte proxy to the maintenance model changes the design from 205 MCV plus 56 FD to 276 MCV plus seven FD. In the shared context, loss falls from 805.316472 to 787.809381, and all seven selected FDs are consumed. For DMV, the 50%, 75%, and 100% budget levels return the same 11-MCV plus 12-FD state, leaving capacity unused because no accepted move improves the objective. Four restricted exhaustive instances recover their exact restricted optima.

Answer to RQ2. CE-Replay drives maintenance-constrained mixed statistics design without a learned design-to-q-error response model. The workload-scale states are audited neighborhood local optima, not global optima.

Table 1: Complementary workload and candidate structure. Degree counts target queries incident to a structural-pair candidate; these workloads are not claimed to represent all database workloads.

Workload	Queries / rows	Predicate columns / form	Pairs	Candidate degree	Incidence	Main stress
Census	468 / 2,458,285	68; numeric equality/range	2,253	Mean 4.44; max 13	Sparse, giant component	Large universe; sparse incremental evaluation
DMV	1,965 / 11,591,877	9; categorical equality and constant IN/= ANY	36	Mean 497.08; median 498; max 546	Dense/high reuse	ScalarArray semantics; dense incremental evaluation

Two-panel maintenance-model placeholder
Census (left) and DMV (right), with environment-specific fits

Figure 2: Mechanism-aware recurring ANALYZE cost. Separate workload panels relate deployed mechanism composition to aggregate ANALYZE latency. Census slopes are 1.874997 ms/MCV and 2.717261 ms/FD; the DMV combined model estimates 3.903845 ms/MCV and 5.907345 ms/FD. Fits and deployment errors are environment-specific, not portable PostgreSQL constants.

Table 2: Native fidelity in the supported fragment. Comparisons match query, design, precedence, and payload realization; error is relative at the raw/native cardinality boundary.

Validation setting	Semantic fragment	Comparisons	Result	Maximum relative error
Census MCV raw oracle	Scalar MCV	Four targets $\times$ 32 designs	All matched	5.55e-16
Controlled FD scenarios	FD selection, application, order	13 scenarios	All matched	0
Synthetic ScalarArray	Constant IN/= ANY MCV	29 cases	29/29; scalar regression 128/128	4.38e-15
DMV baseline realization	Real equality/IN MCV+FD	27,510	27,510/27,510	1.40286e-14
Census fresh deployment	Fresh mixed MCV+FD	468	468/468	6.92e-16
DMV fresh deployment	Fresh mixed MCV+FD	1,965	1,965/1,965	2.43422e-14

8.3 RQ3 — Do semantic dependencies reduce move-evaluation work?

RQ3 asks both whether incremental evaluation preserves decisions and whether less semantic work becomes lower runtime. **Table 4** keeps exactness, semantic/control-work reduction, and wall-clock behavior separate across sparse and dense incidence.

Census contains a giant component despite sparse degree, so connected-component decomposition is ineffective. Local invalidation remains useful because a typical move touches few queries.

The MCV-only evaluator preserves its complete accepted trajectory and improves measured phase time. The mixed evaluator also preserves its trajectory and sharply reduces control work, yet it is slower because numerical aggregation and Python move-loop overhead dominate. DMV shows that dependency-aware evaluation still avoids work under dense incidence, although the opportunity is smaller.

Answer to RQ3. Realized and structural/counterfactual dependencies enable exact audited incremental move evaluation in sparse and dense regimes. Topology controls the opportunity, and

Two-panel non-monotonicity placeholder
Census (left) and DMV (right), with independent raw-loss scales

Figure 3: Cross-workload non-monotonicity. Separate, independently scaled panels compare empty and complete states. Census has 1,560 harmful singleton additions among 3,011 candidates and 317 improving removals; DMV has 45 harmful additions among 70 usable candidates and 17 improving removals. This is contextual evidence from two frozen realizations, not a universal workload property.

Table 3: Maintenance-budget design outcomes. Local optimum is limited to the audited ADD/DROP/SWAP neighborhood under fixed payload and precedence.

Workload	Usable universe	Selected design	Budget; cost / unused	Frozen loss	Terminal audit
Census	2,253 MCV + 758 FD	276 MCV + 7 FD	286.144; 286.143 / 0.001	787.809381	565,031 moves; local optimum
DMV optimization realization	36 MCV + 34 FD	11 MCV + 12 FD	43.724603; 29.158543 / 14.566060	7.83651388676e298	47 ADD, 23 DROP, 1,081 SWAP; local optimum

Table 4: Exact incremental evaluation across incidence regimes. Semantic work and wall-clock outcomes remain separate.

Setting	Incidence	Audited exactness	Semantic-work reduction	Wall clock
Census MCV local search	Sparse; mean degree 4.44	5,639,186 moves; 17 accepted and final state identical	171.31× fewer query control replays; 95.28% no control replay	82.422 s vs 289.013 s; 3.51×
Census mixed local search	Sparse; directed MCV→FD	9,107,766 moves; 19 accepted and final state identical	95.13% numerical-only; 154.61× control reduction	145.68 s vs 45.05 s; 3.23× slower
DMV mixed optimization	Dense; mean degree 497.13	1,151 terminal moves; four restricted exact recoveries	67.91% full-workload query replay operations avoided	No comparable speedup claim

semantic-work reduction does not guarantee wall-clock improvement.

8.4 RQ4 – Do interacting mechanisms survive deployment?

RQ4 asks whether the directed MCV-to-FD interaction survives complete design and physical deployment. **Figure 4** links that mechanism to observed FD reachability.

Joint Census optimization improves over independently optimized mechanisms and stops spending maintenance capacity on many suppressed FDs. DMV replicates the interaction under dense incidence. **Table 5** then separates physical realization, fresh same-realization fidelity, and frozen-to-fresh drift.

For Census, all objects materialize and consume. Across 30 repeated ANALYZE realizations, 14,040/14,040 replay/native comparisons match while payloads and objective values vary. This demonstrates same-realization fidelity under payload variation, not stability of design rankings. For DMV, one selected FD payload does not materialize, but every materialized FD consumes and all fresh queries match native. The original optimization artifact lacks sufficient frozen per-query state for a paired DMV drift distribution; this provenance gap does not invalidate fresh semantic fidelity or physical composition.

Answer to RQ4. CE-Replay composes directed MCV and FD semantics and remains faithful after fresh physical deployment on both workloads. Fresh realization can change values and availability,

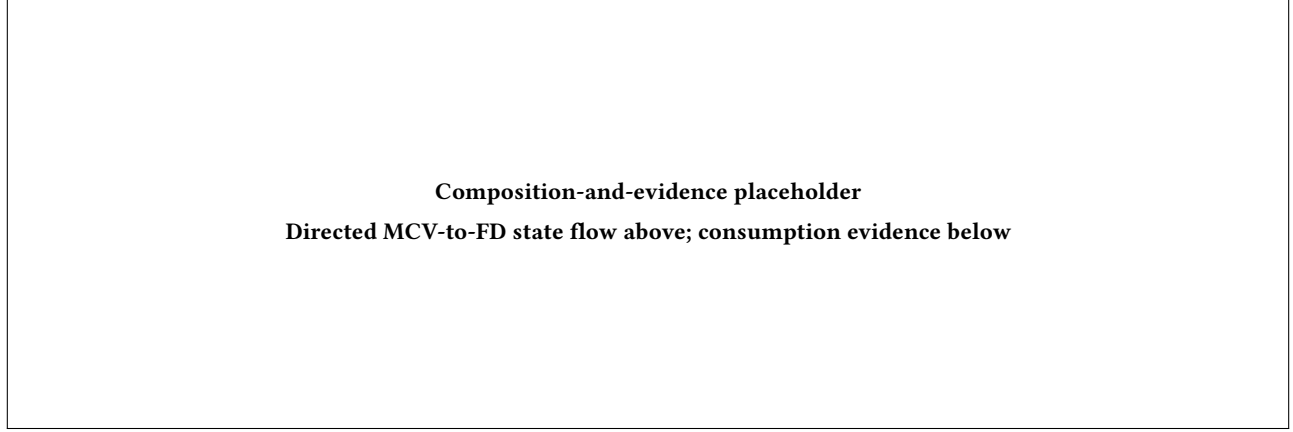


Figure 4: Directed MCV-to-FD composition and FD consumption. MCV clause consumption feeds FD residual state. Census independent optimization selects 97 FDs, 72 unused after composition; joint optimization selects 54, all consumed. DMV all-statistics consumes zero of 34 usable FDs; optimization consumes all 12 selected frozen FDs and all 11 materialized fresh FDs.

Table 5: Physical deployment and fresh validation. Unavailable DMV drift reflects missing frozen provenance, not assumed zero drift.

Workload	Selected / fresh materialized	FD consumption	Fresh replay/native; max relative error	Frozen→fresh drift	Cost error
Census	276 MCV + 7 FD; 276/276 MCV, 7/7 FD	7/7	811.553725 / 811.553725; 468/468; 6.92e-16	787.809381 → 811.553725; +3.0140%	7.20%
DMV	11 MCV + 12 FD; 11/11 MCV, 11/12 FD	11/11 materialized; 389 queries	1,965/1,965; aggregate equal; 2.43422e-14	Unavailable: frozen per-query provenance not persisted	21.4295%

so semantic replay error and payload realization drift must remain separate.

9 DISCUSSION AND LIMITATIONS

9.1 Statistics as semantic physical design

Extended statistics change the optimizer’s information state, and their utility depends on native selection, consumption, and mechanism composition—not only on captured correlation. The cross-workload non-monotonicity evidence therefore justifies selection independently of scarcity, while recurring collection work supplies a separate resource justification. A loose budget removes neither contextual harm nor the need to choose.

9.2 CE-Replay as an optimization representation

CE-Replay’s value is representational rather than merely computational. Its executable state transitions provide both the hypothetical objective and the information needed to invalidate that objective safely after a move. Within the supported fragment, native semantics supply the design response without a separately learned subset-to-error model; outside that boundary, approximate or learned components may still be useful.

Execution-sufficient state is not necessarily counterfactual-safe state. A currently unused object may change a future winner, and numerically equal current states can diverge after an extension. This is why structural/counterfactual dependencies supplement the realized trace.

9.3 Locality without decomposition

Census rejects a tempting but incorrect shortcut. Its graph is globally connected and tested factorization does not yield independent subproblems, yet sparse candidate degree permits local recomputation. The semantic representation reduces exact move-evaluation work; it does not decompose away the global budget or combinatorial search.

The mixed-evaluator slowdown identifies the limit of that opportunity. Once semantic control replay becomes rare, numerical aggregation and implementation overhead dominate; less replay work does not guarantee end-to-end acceleration. DMV confirms correctness in a dense regime while exposing less locality, but two workloads do not define a universal scaling law.

9.4 Maintenance cost and semantic utility

Payload bytes were an early controlled proxy. The final resource is recurring maintenance, instantiated through mechanism-weighted

object counts calibrated from aggregate ANALYZE latency. Coefficients differ across Census and DMV and deployment prediction errors remain visible. The model is a first-order environment-specific resource proxy, not a PostgreSQL constant or a per-object latency predictor.

Changing resource semantics changes the physical-design problem: on Census it materially changes composition and frozen loss. A budget is a capacity constraint, not a target; a local optimum may leave capacity unused when no feasible neighborhood move improves the contextual objective.

9.5 Frozen hypothetical state and fresh realization

The optimizer is conditional on a frozen payload repository, while deployment generates a fresh realization. Replay/native fidelity within one realization and frozen-to-fresh drift answer different questions. Repeated Census ANALYZE demonstrates payload and objective variability without semantic mismatch. It does not establish frequent design-ranking reversal or regret; that would require paired multi-design realizations.

Future objectives could optimize expected, risk-sensitive, or worst-case loss over realizations. Efficient acquisition of hypothetical candidate payloads is another open systems problem. Just-in-time, piggyback, and modern incremental statistics maintenance demonstrate complementary ways to reduce collection or refresh work [5, 11, 15]. CE-Replay does not implement those techniques: its offline acquisition boundary is distinct from the recurring maintenance cost of the selected deployment.

9.6 Extending CE-Replay

The implementation is source-informed and mechanism-specific rather than an automatic compiler. Supporting another predicate family, mechanism, version, or DBMS requires extracting applicability and control rules, defining payload schemas, implementing state transitions, validating native behavior, and exposing dependencies. The ScalarArray extension demonstrates this deliberate process. MCV-to-FD composition suggests an architectural principle—mechanisms should expose explicit state inputs, outputs, and dependency boundaries—but the principle has not been validated across DBMSs.

PostgreSQL-specific elements include GreedyCover, OID precedence, payload formats, MCV-to-FD ordering, and ANALYZE realization behavior. The broader abstractions are workload specialization, design-parametric executable semantics, dual oracles, realization boundaries, and maintenance-constrained design.

9.7 Limitations

The validated semantic scope is PostgreSQL 16.14 conjunctive base-relation restrictions with supported scalar and constant IN/= ANY predicates, MCV, FD, and their directed composition. It excludes joins, planner/path search, arbitrary expressions/operators or Boolean trees, other mechanisms, and other versions. The semantics were manually engineered and validated, not automatically compiled.

Optimization assumes an offline frozen payload repository and returns an ADD/DROP/SWAP neighborhood local optimum under fixed precedence; it has no full-instance global or approximation guarantee. The maintenance model is environment-specific and first-order. The objective is q-error, not plan quality, latency, throughput, or a causal runtime improvement.

Fresh ANALYZE can change payloads or availability, and the optimizer is not realization-robust. DMV raw aggregate loss is dominated by two zero-truth queries under the preserved positive floor. DMV also lacks sufficient frozen per-query provenance for complete paired drift reconstruction, although fresh fidelity and composition remain supported.

Finally, Census and DMV provide complementary sparse and dense evidence but do not establish universal workload generality. The target workload is supplied; unseen-workload generalization is outside the core problem.

10 RELATED WORK

10.1 Statistics management and multivariate-statistics discovery

Automatic statistics management predates CE-Replay. Chaudhuri and Narasayya identify optimizer-relevant statistics and reduce the set that must be created for a workload [4]. CORDS discovers correlations and soft functional dependencies and recommends column groups for joint statistics [7]. JITS selects and collects query-specific statistics during compilation using sensitivity and previously collected information [5]. These works establish workload-aware statistics selection, correlation discovery, and selective acquisition as existing problem classes.

Within their MNSA formulation for SPJ queries, Chaudhuri and Narasayya use a cost-monotonicity assumption over predicate selectivity variables to bound optimizer-cost sensitivity [4]. The non-monotonicity evaluated here instead concerns PostgreSQL q-error under inclusion of physically realized extended-statistics objects. The DBMS, objective, varied state, and estimator semantics differ, so this is a contrast between problem structures rather than a contradiction of the earlier assumption.

CE-Replay does not claim candidate discovery or acquisition as a contribution. It assumes candidate definitions and a frozen payload repository, then evaluates complete candidate subsets through the supported native consumption semantics. Its focus is the interaction among already proposed statistics definitions: which objects become applicable, which win, which clauses they consume, and which downstream mechanisms remain reachable.

Statistics collection and refresh are also established concerns. Piggyback collection gathers statistics during query execution [15], while JITS considers materialization and maintenance of selectively collected statistics [5]. Recent Redshift work updates optimizer statistics incrementally from modified data rather than repeatedly scanning full tables [11]. These techniques ask how to acquire or refresh a chosen statistical state efficiently. CE-Replay asks which state to choose under a recurring maintenance constraint; combining the two directions is future systems work.

10.2 What-if and configuration-parametric physical design

Hypothetical evaluation and constrained design are established foundations. AutoAdmin’s What-If utility evaluates hypothetical indexes without materializing every configuration [3]. Constrained physical-design tuning formulates workload optimization under storage and richer design constraints [1]. CE-Replay therefore does not claim novelty for resource-constrained physical design or hypothetical evaluation in isolation.

INUM caches and reuses optimizer-derived plan information to estimate query cost across index configurations with far fewer optimizer calls [10]. C-PQO constructs a compact representation of optimizer behavior that can produce plans for arbitrary physical configurations after a specialized optimizer invocation [2]. These systems are close conceptual neighbors: they show that avoiding repeated optimizer calls and constructing configuration-parametric representations are established ideas.

The distinction is the represented computation. INUM and C-PQO represent optimizer plan/cost behavior over hypothetical physical-design configurations. CE-Replay specializes the supported statistics-sensitive estimator computation itself, leaving applicability, winner selection, clause consumption, payload-dependent numerical updates, and cross-mechanism state executable. This difference supports a statistics-design objective and dependencies internal to the CE computation; it is not a claim that prior physical-design tools are black boxes or lack response reuse.

10.3 Incremental optimization

Dependency-aware incremental recomputation is not new in general. Incremental query re-optimization maintains search and pruning state as cardinality or cost information changes [9]. CE-Replay operates at a different layer: a statistics-design move propagates through estimator semantics and changes CE and workload loss before downstream optimizer search. We therefore use the narrower term **semantic dependency oracle** rather than claim generic incremental optimization as a contribution.

10.4 Cardinality estimation

Learned and data-driven CE systems construct alternative estimation models. MSCN learns from query features and executed-query cardinalities [8]; DeepDB learns a data-driven probabilistic model rather than requiring executed-query workload training [6]; FactorJoin combines learned single-table conditional distributions through a factor graph for join estimation [14]. This variety prevents a simple learning-versus-no-learning dichotomy.

CE-Replay does not learn a replacement cardinality estimator or a surrogate mapping from statistics designs to q-error. It executes the supported statistics-sensitive semantics of PostgreSQL’s native estimator while varying the physical statistics state. PostgreSQL documentation and source define the native MCV/FD mechanisms [12, 13]; this paper contributes their bounded extraction, executable encoding, composition, and validation for physical-design evaluation, not the mechanisms themselves.

10.5 Positioning of CE-Replay

Prior work addresses statistics discovery and management, hypothetical and configuration-parametric physical-design evaluation, incremental optimizer maintenance, and alternative cardinality estimators. CE-Replay focuses on the interface between statistics physical design and the supported statistics-sensitive native CE computation. It makes that computation executable under hypothetical statistics states and uses the same representation both to evaluate workload CE loss and to expose statistics-semantic dependencies for design moves.

This positioning is deliberately bounded. The literature categories overlap, and the current search does not justify a priority claim. CE-Replay is neither a complete advisor pipeline nor a full PostgreSQL estimator: candidate-payload acquisition remains external, search remains replaceable and neighborhood-local, and semantic coverage remains the validated PostgreSQL 16.14 fragment.

11 CONCLUSION

This paper studies extended-statistics design for a supplied target workload under a recurring maintenance constraint. Selection is necessary both because deployed objects consume maintenance capacity and because native estimator control makes candidate utility contextual and non-monotone.

CE-Replay exposes the supported statistics-sensitive semantics as a workload-specialized, design-parametric executable representation. Workload-fixed context is specialized, while applicability, winner selection, clause consumption, MCV-to-FD composition, and numerical payload behavior remain executable. The resulting objective and dependency oracles allow a replaceable optimizer to evaluate hypothetical states and safely restrict recomputation.

For the supported PostgreSQL 16.14 base-restriction fragment, Census and DMV establish same-realization fidelity within floating-point tolerance, maintenance-aware mixed designs, audited incremental equivalence in sparse and dense regimes, and fresh physical deployment. The negative results delimit the method: global connectivity need not eliminate local invalidation, less semantic replay does not guarantee faster optimization, and fresh payload drift remains distinct from semantic replay error.

The current results remain conditional on a supplied workload, a frozen payload repository, manually supported MCV+FD semantics, and neighborhood-local search. Extending executable semantic representations to broader CE mechanisms, cheaper payload acquisition, realization-aware objectives, and additional DBMSs offers a path toward physical-design tools that use native estimator behavior as both an objective evaluator and a source of optimization structure.

ARTIFACT AND REPRODUCIBILITY STATEMENT

The public repository contains experiment scripts, frozen results, and claim/evidence audits. The immutable research-freeze-v0 tag identifies the experimental baseline used by this draft; later commits contain paper-construction artifacts. The reproducibility index maps each main claim to its scripts and outputs. No archival DOI is claimed in this draft.

REFERENCES

- [1] Nicolas Bruno and Surajit Chaudhuri. 2008. Constrained Physical Design Tuning. *Proceedings of the VLDB Endowment* 1, 1 (2008), 4–15. <https://doi.org/10.14778/1453856.1453857>
- [2] Nicolas Bruno and Rimma V. Nehme. 2008. Configuration-Parametric Query Optimization for Physical Design Tuning. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. 941–952. <https://doi.org/10.1145/1376616.1376710>
- [3] Surajit Chaudhuri and Vivek R. Narasayya. 1998. AutoAdmin “What-If” Index Analysis Utility. In *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data*. 367–378. <https://doi.org/10.1145/276304.276337>
- [4] Surajit Chaudhuri and Vivek R. Narasayya. 2001. Automating Statistics Management for Query Optimizers. *IEEE Transactions on Knowledge and Data Engineering* 13, 1 (2001), 7–20. <https://doi.org/10.1109/69.908978>
- [5] Amr El-Helw, Ihab F. Ilyas, Wing Lau, Volker Markl, and Calisto Zuzarte. 2007. Collecting and Maintaining Just-in-Time Statistics. In *Proceedings of the 23rd IEEE International Conference on Data Engineering*. <https://doi.org/10.1109/ICDE.2007.367897>
- [6] Benjamin Hilprecht, Andreas Schmidt, Moritz Kulessa, Alejandro Molina, Kristian Kersting, and Carsten Binnig. 2020. DeepDB: Learn from Data, not from Queries! *Proceedings of the VLDB Endowment* 13, 7 (2020), 992–1005. <https://doi.org/10.14778/3384345.3384349>
- [7] Ihab F. Ilyas, Volker Markl, Peter J. Haas, Paul Brown, and Ashraf Aboulmaga. 2004. CORDS: Automatic Discovery of Correlations and Soft Functional Dependencies. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*. 647–658. <https://doi.org/10.1145/1007568.1007641>
- [8] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *9th Biennial Conference on Innovative Data Systems Research*. <https://www.cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf>
- [9] Mengmeng Liu, Zachary G. Ives, and Boon Thau Loo. 2016. Enabling Incremental Query Re-Optimization. In *Proceedings of the 2016 ACM SIGMOD International Conference on Management of Data*. 1705–1720. <https://doi.org/10.1145/2882903.2915212>
- [10] Stratos Papadomanolakis, Debabrata Dash, and Anastasia Ailamaki. 2007. Efficient Use of the Query Optimizer for Automated Physical Design. In *Proceedings of the 33rd International Conference on Very Large Data Bases*. 1093–1104. <https://www.vldb.org/conf/2007/papers/research/p1093-papadomanolakis.pdf>
- [11] Pascal Pfeil, Zhe Heng Eng, Magnus Mueller, George Erickson, Roger Kim, Mohammed Al-Kateb, Majid Saeedan, Dominik Horn, Orestis Polychroniou, Mengchu Cai, and Tim Kraska. 2026. Incremental Query Optimizer Statistics in Amazon Redshift. (2026). <https://www.amazon.science/publications/incremental-query-optimizer-statistics-in-amazon-redshift> VLDB 2026.
- [12] PostgreSQL Global Development Group. 2023. PostgreSQL 16 Documentation: Multivariate Statistics Examples. <https://www.postgresql.org/docs/16/multivariate-statistics-examples.html> Version 16 documentation.
- [13] PostgreSQL Global Development Group. 2025. PostgreSQL 16.14 Source: Extended Statistics Selectivity. Source files `src/backend/statistics/extended_stats.c`, `mcv.c`, and `dependencies.c`. https://github.com/postgres/postgres/tree/REL_16_14/src/backend/statistics
- [14] Ziniu Wu, Parimarjan Negi, Mohammad Alizadeh, Tim Kraska, and Samuel Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *Proceedings of the ACM on Management of Data* 1, 1, Article 41 (2023), 27 pages. <https://doi.org/10.1145/3588721>
- [15] Qiang Zhu, Brian Dunkel, Wing Lau, Suyun Chen, and Berni Schiefer. 2004. Piggyback Statistics Collection for Query Optimization: Towards a Self-Maintaining Database Management System. *Comput. J.* 47, 2 (2004), 221–244. <https://doi.org/10.1093/comjnl/47.2.221>